

```
In [1]: import seaborn as sns
print(sns.get_dataset_names())
```

```
['anagrams', 'anscombe', 'attention', 'brain_networks', 'car_crashes', 'diamonds',
'dots', 'dowjones', 'exercise', 'flights', 'fmri', 'geyser', 'glue', 'healthexp', 'i
ris', 'mpg', 'penguins', 'planets', 'seaice', 'taxis', 'tips', 'titanic']
```

```
In [2]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

1. Penguins

```
In [3]: # Load the dataset first (This defines 'df')
df = sns.load_dataset("penguins")
df
```

```
Out[3]:
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0
3	Adelie	Torgersen	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0
...	...	...	...	...	...	...
339	Gentoo	Biscoe	NaN	NaN	NaN	NaN
340	Gentoo	Biscoe	46.8	14.3	215.0	4850.0
341	Gentoo	Biscoe	50.4	15.7	222.0	5750.0
342	Gentoo	Biscoe	45.2	14.8	212.0	5200.0
343	Gentoo	Biscoe	49.9	16.1	213.0	5400.0

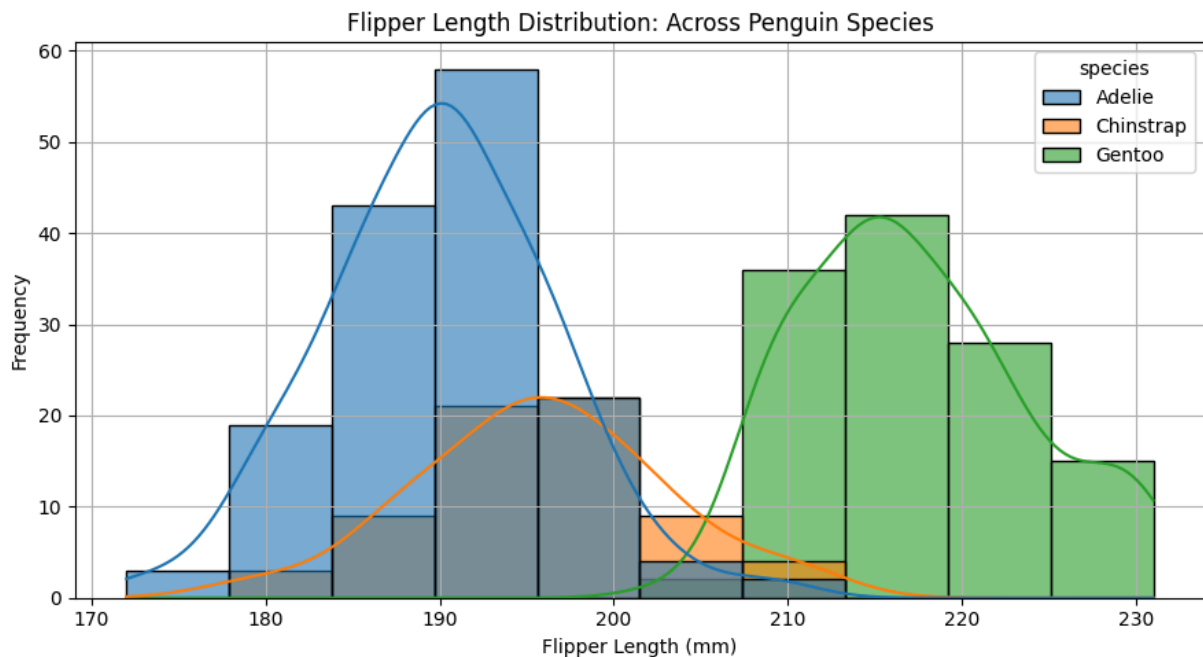
344 rows × 7 columns



Histogram

```
In [4]: plt.figure(figsize=(9,5))
sns.histplot(data=df, x='flipper_length_mm', hue='species', kde=True, multiple='lay
plt.title('Flipper Length Distribution: Across Penguin Species')
plt.xlabel('Flipper Length (mm)')
plt.ylabel('Frequency')
```

```
plt.grid(True)
plt.tight_layout()
plt.show()
```

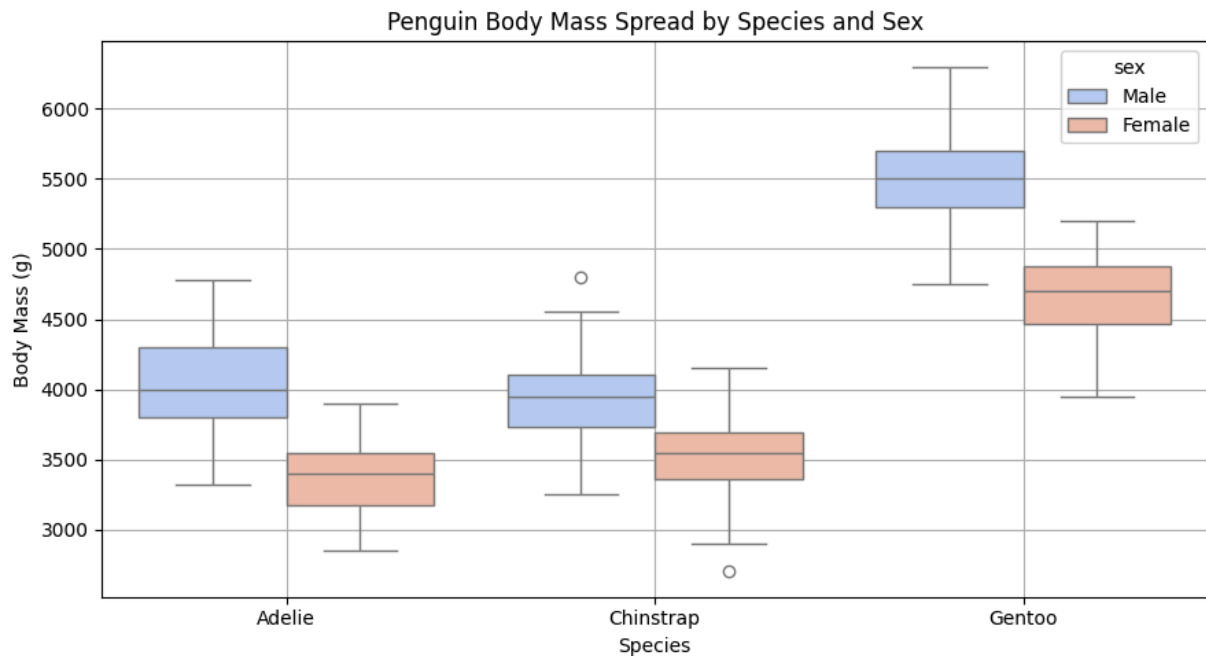


This code creates a layered histogram with smooth trend lines to compare the distribution of flipper lengths across different penguin species.

In []:

Box Plot

```
In [5]: plt.figure(figsize=(9,5))
sns.boxplot(data=df, x='species', y='body_mass_g', hue='sex', palette='coolwarm')
plt.title('Penguin Body Mass Spread by Species and Sex')
plt.xlabel('Species')
plt.ylabel('Body Mass (g)')
plt.grid(True)
plt.tight_layout()
plt.show()
```

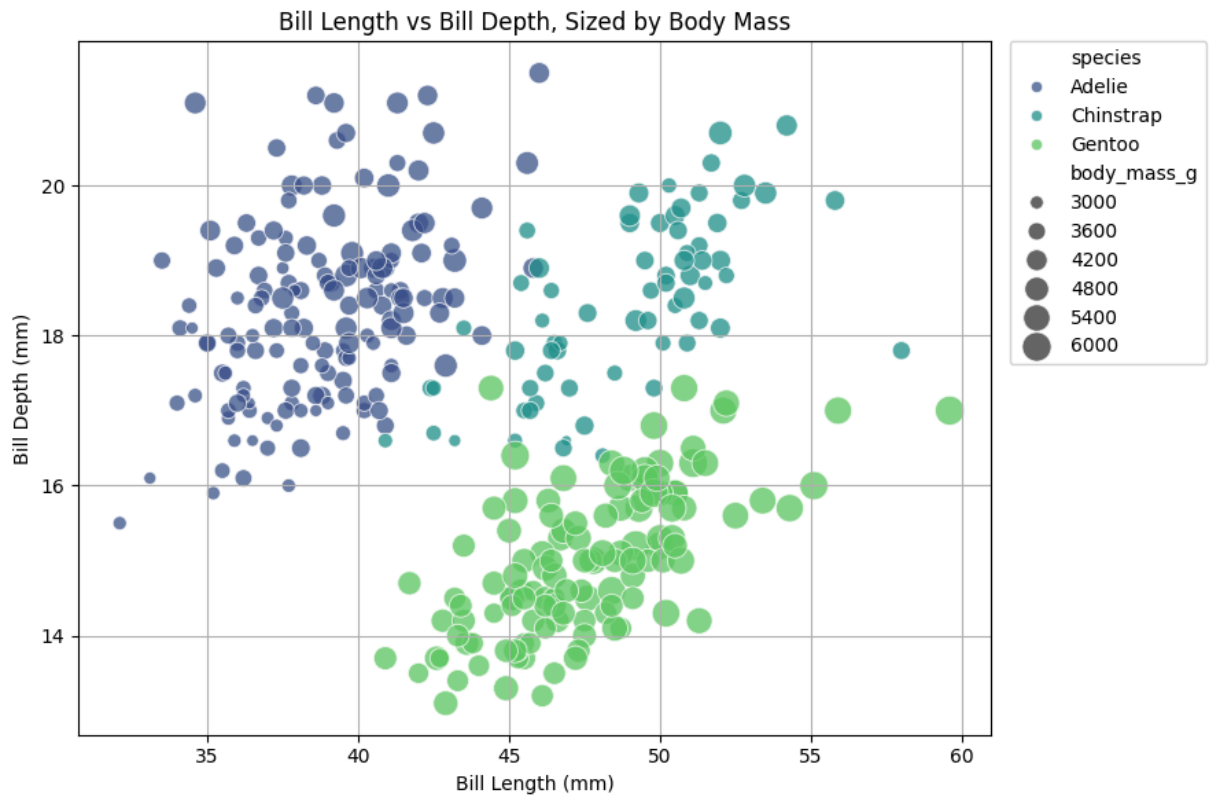


This code creates a grouped box plot to compare the distribution, medians, and ranges of penguin body mass across different species, further broken down by sex

In []:

Scatter Plot

```
In [6]: plt.figure(figsize=(9,6))
sns.scatterplot(data=df, x='bill_length_mm', y='bill_depth_mm', hue='species', size=
plt.title('Bill Length vs Bill Depth, Sized by Body Mass')
plt.xlabel('Bill Length (mm)')
plt.ylabel('Bill Depth (mm)')
plt.legend(bbox_to_anchor=(1.02, 1), loc='upper left', borderaxespad=0)
plt.grid(True)
plt.tight_layout()
plt.show()
```



This code creates a multivariate scatter plot to analyze the relationship between bill length and bill depth across species, while representing body mass through point sizes.

In []:

In []:

2. Titanic

```
In [7]: # Load the dataset first (This defines 'df')
df = sns.load_dataset("titanic")
df
```

Out[7]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adl
0	0	3	male	22.0	1	0	7.2500	S	Third	man	
1	1	1	female	38.0	1	0	71.2833	C	First	woman	
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	
3	1	1	female	35.0	1	0	53.1000	S	First	woman	
4	0	3	male	35.0	0	0	8.0500	S	Third	man	
...	...	...	...	...	...	...	...	...	...	...	...
886	0	2	male	27.0	0	0	13.0000	S	Second	man	
887	1	1	female	19.0	0	0	30.0000	S	First	woman	
888	0	3	female	NaN	1	2	23.4500	S	Third	woman	
889	1	1	male	26.0	0	0	30.0000	C	First	man	
890	0	3	male	32.0	0	0	7.7500	Q	Third	man	

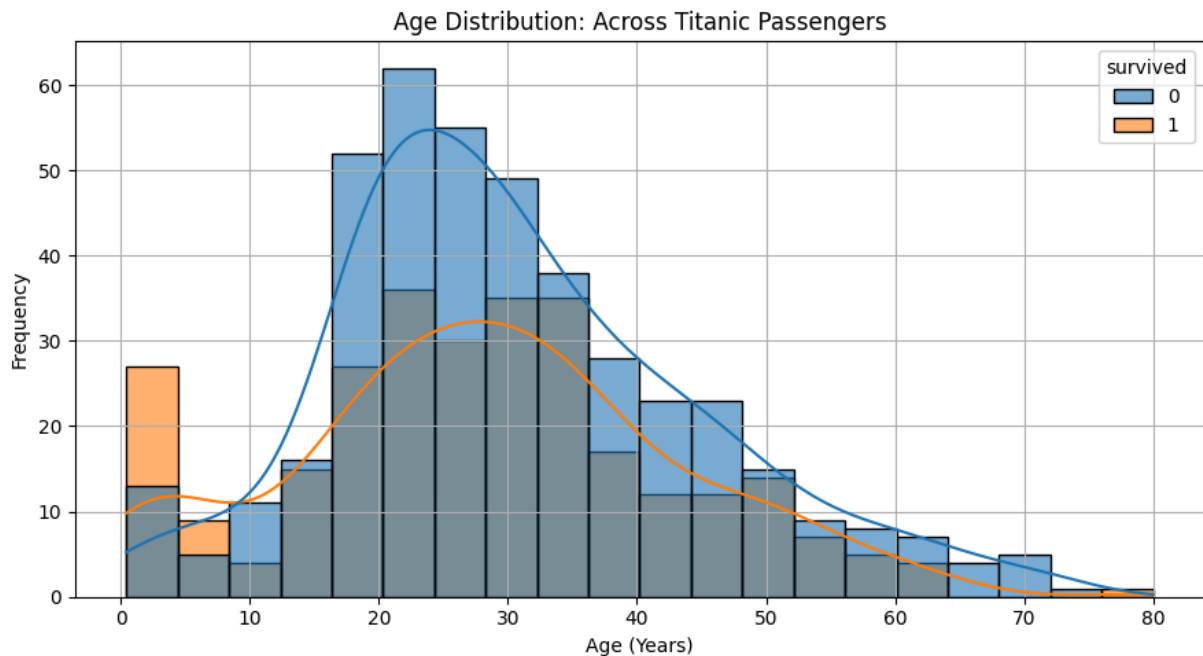
891 rows × 15 columns



 Histogram

In [8]:

```
plt.figure(figsize=(9, 5))
sns.histplot(data=df, x="age", hue="survived", kde=True, multiple="layer", alpha=0.5)
plt.title("Age Distribution: Across Titanic Passengers")
plt.xlabel("Age (Years)")
plt.ylabel("Frequency")
plt.grid(True)
plt.tight_layout()
plt.show()
```



This code creates a layered histogram with smooth trend lines to compare the frequency and distribution of passenger ages based on whether they survived or died on the Titanic.

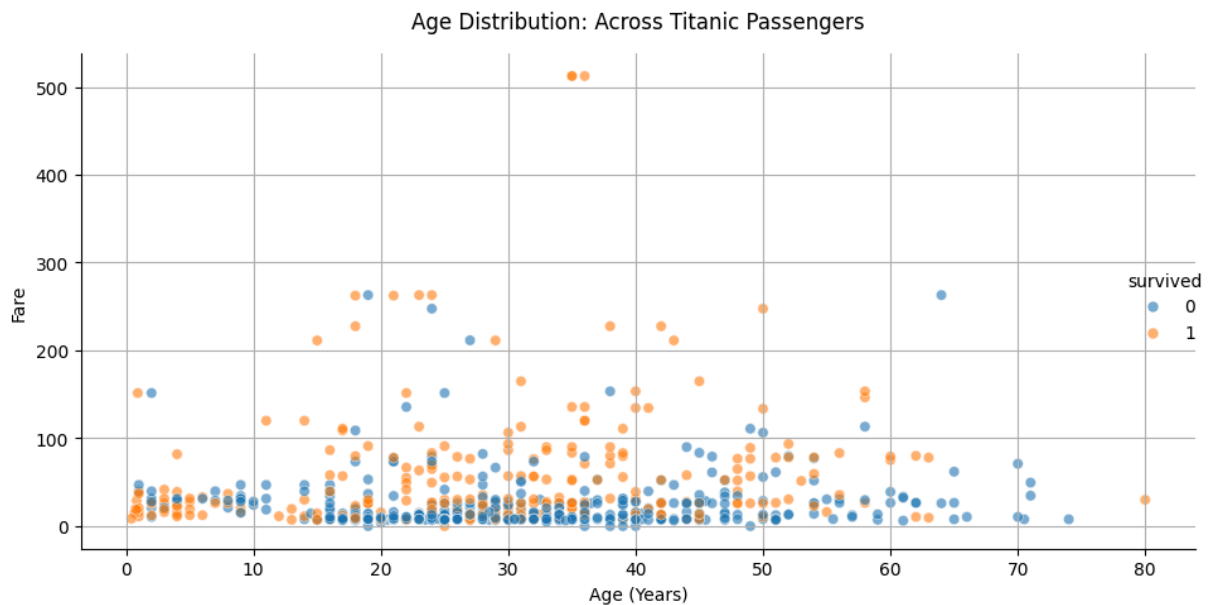
In []:



Relplot

```
In [9]: plt.figure(figsize=(9, 5))
sns.relplot(data=df, x="age", y="fare", hue="survived", kind="scatter", alpha=0.6, height=1.03)
plt.title("Age Distribution: Across Titanic Passengers", y=1.03)
plt.xlabel("Age (Years)")
plt.ylabel("Fare")
plt.grid(True)
plt.tight_layout()
plt.show()
```

<Figure size 900x500 with 0 Axes>



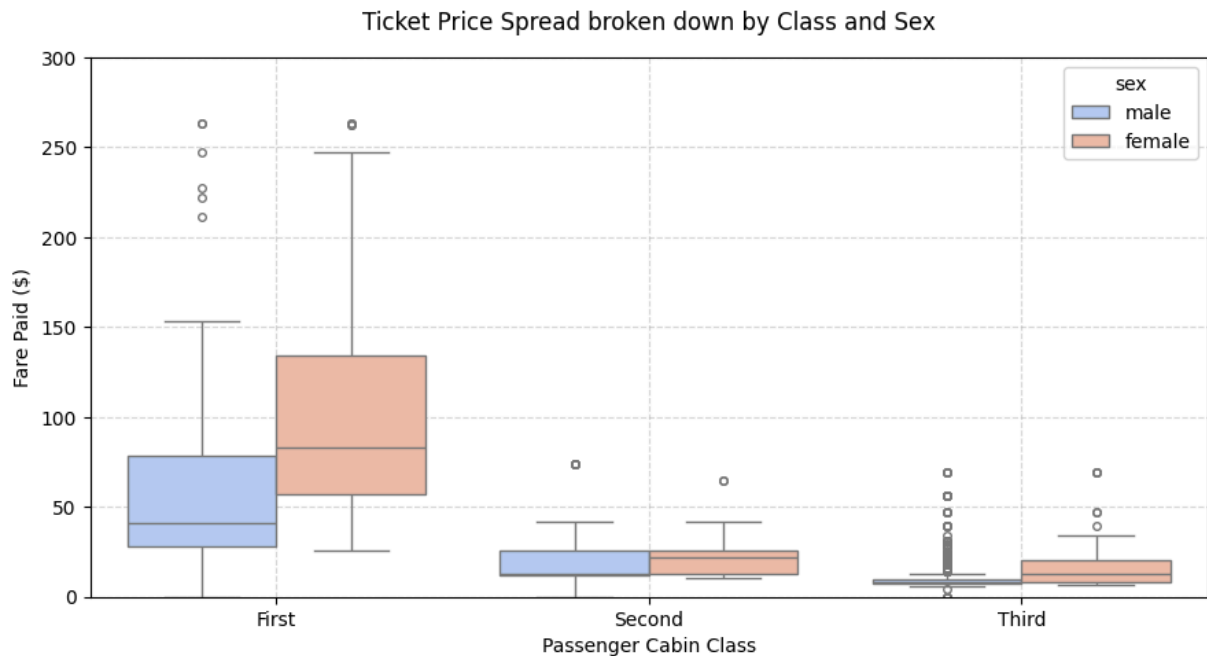
This code creates a multivariate scatter plot to analyze the relationship between passenger age and ticket fare, color-coded by survival status across Titanic passengers.

In []:



Boxplot

```
In [11]: plt.figure(figsize=(9, 5))
sns.boxplot(data=df, x="class", y="fare", hue="sex", palette="coolwarm", fliersize=
plt.title("Ticket Price Spread broken down by Class and Sex", y=1.03)
plt.xlabel("Passenger Cabin Class")
plt.ylabel("Fare Paid ($)")
plt.ylim(0, 300) # Caps excessive outliers to keep the grid clear
plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()
```



This code creates a grouped box plot to compare the distribution, median, and outlier ranges of ticket fares paid across different cabin classes, further broken down by passenger sex.

In []:

In []:

3 Diamonds

```
In [12]: # Load the dataset first (This defines 'df')
df = sns.load_dataset("diamonds")
df
```

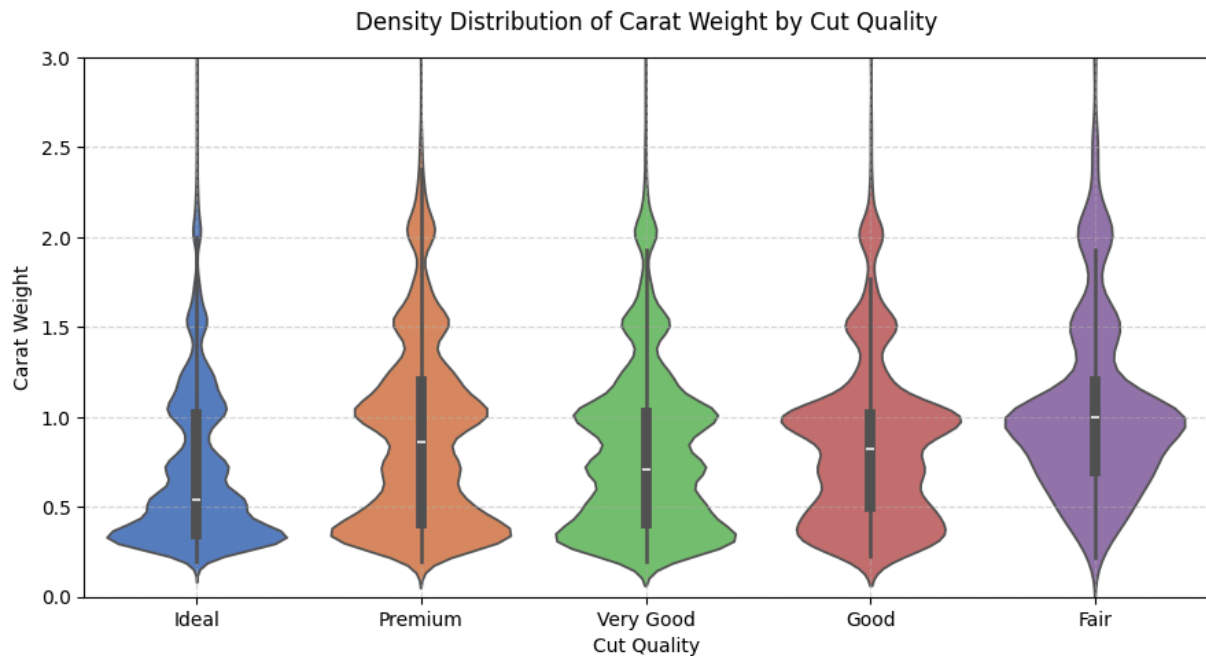

Out[12]:

	carat	cut	color	clarity	depth	table	price	x	y	z
0	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
1	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
2	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
3	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
4	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75
...	...	...	...	...	...	...	...	...	...	...
53935	0.72	Ideal	D	SI1	60.8	57.0	2757	5.75	5.76	3.50
53936	0.72	Good	D	SI1	63.1	55.0	2757	5.69	5.75	3.61
53937	0.70	Very Good	D	SI1	62.8	60.0	2757	5.66	5.68	3.56
53938	0.86	Premium	H	SI2	61.0	58.0	2757	6.15	6.12	3.74
53939	0.75	Ideal	D	SI2	62.2	55.0	2757	5.83	5.87	3.64

53940 rows × 10 columns

 Violinplot

```
In [15]: plt.figure(figsize=(9, 5))
sns.violinplot( data=df, x="cut", y="carat", hue="cut", legend=False, palette="muted")
plt.title("Density Distribution of Carat Weight by Cut Quality", y=1.03)
plt.xlabel("Cut Quality")
plt.ylabel("Carat Weight")
plt.ylim(0, 3)
plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()
```

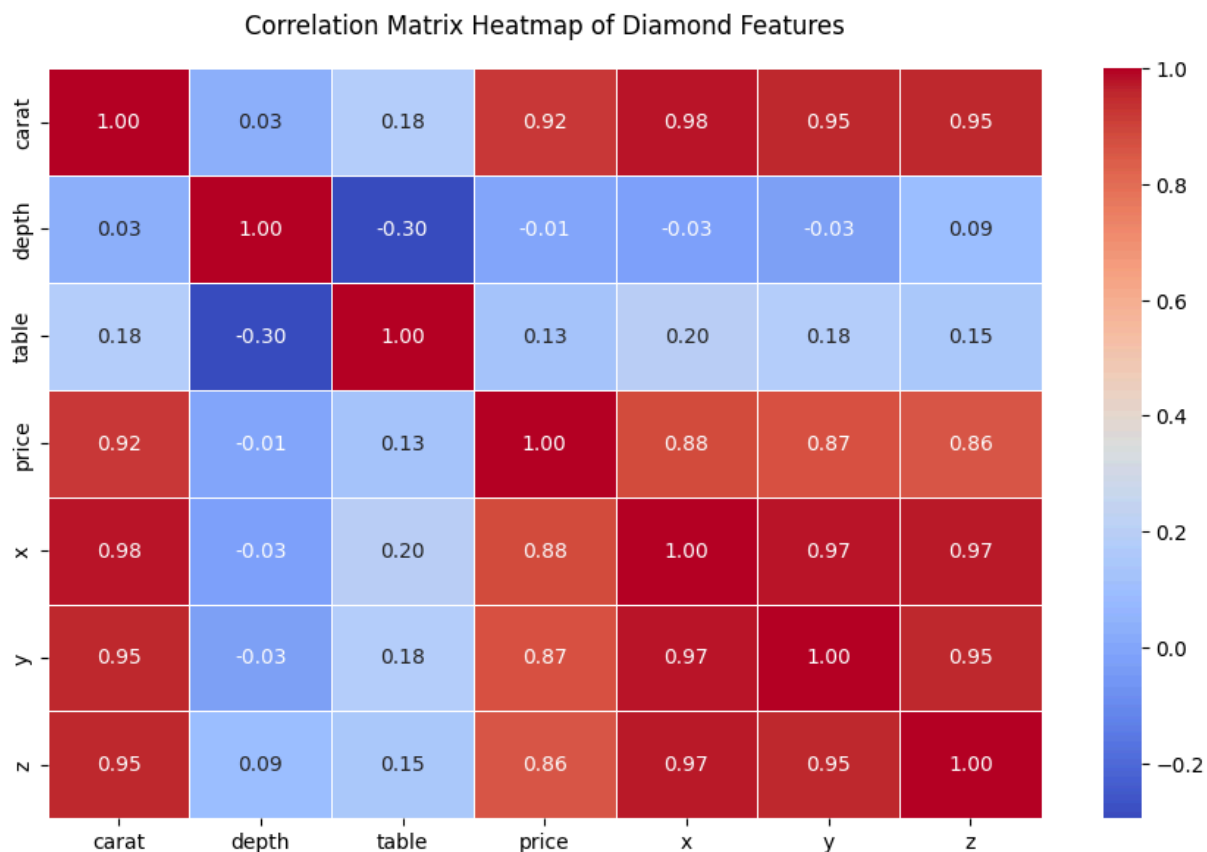


This code creates a violin plot with internal box plots to compare the probability density and weight distribution of diamond carats across different cut qualities.

In []:

Heatmap

```
In [17]: plt.figure(figsize=(9, 6))
numerical_df = df.select_dtypes(include=["float64", "int64"])
corr_matrix = numerical_df.corr()
sns.heatmap(data=corr_matrix,annot=True, cmap="coolwarm", fmt=".2f", linewidths=0.5
plt.title("Correlation Matrix Heatmap of Diamond Features", y=1.03)
plt.tight_layout()
plt.show()
```

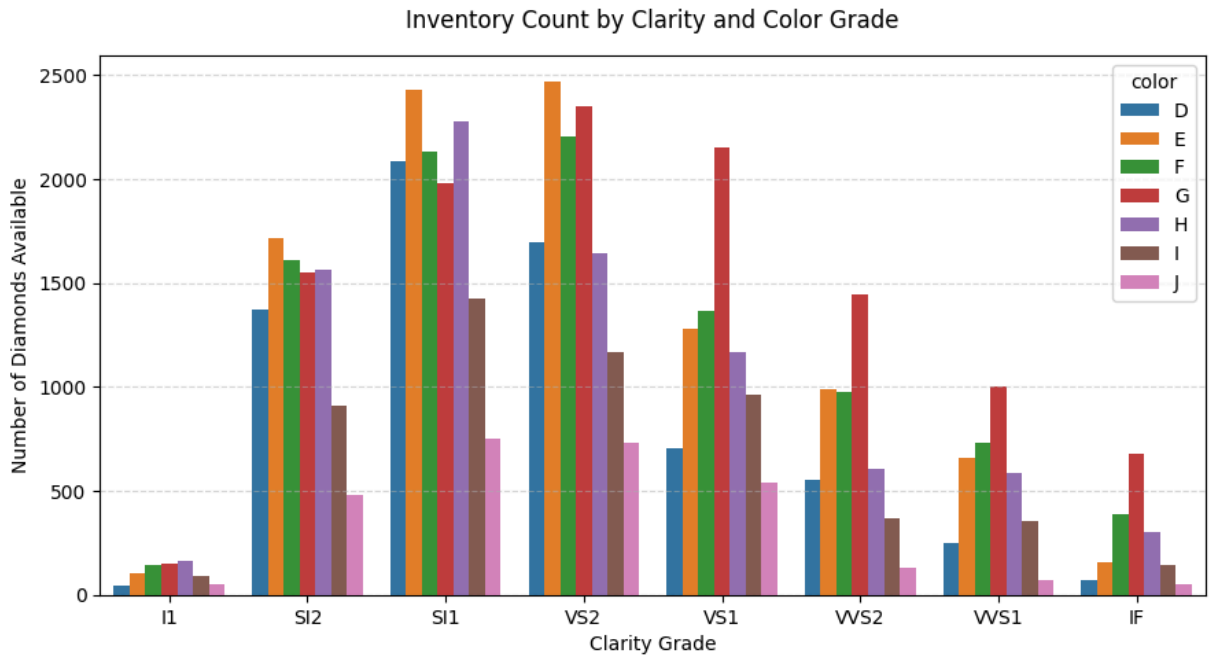


This code creates an annotated heatmap matrix to measure and visually check the strength of linear relationships between all numerical features of the diamonds.

In []:

Countplot

```
In [19]: plt.figure(figsize=(9, 5))
          clarity_order = ["I1", "SI2", "SI1", "VS2", "VS1", "VVS2", "VVS1", "IF"]
          sns.countplot( data=df, x="clarity", hue="color", order=clarity_order, palette="tab
          plt.title("Inventory Count by Clarity and Color Grade", y=1.03)
          plt.xlabel("Clarity Grade")
          plt.ylabel("Number of Diamonds Available")
          plt.grid(True, axis="y", linestyle="--", alpha=0.5)
          plt.tight_layout()
          plt.show()
```



This code creates a grouped count plot to compare the inventory volume of diamonds across ordered clarity grades, further broken down by color categories.

In []:

In []:

4 Taxis 🚕

```
In [21]: # Load the dataset first (This defines 'df')
df = sns.load_dataset("taxi")
df
```

Out[21]:

	pickup	dropoff	passengers	distance	fare	tip	tolls	total	color	payment	pi
0	2019-03-23 20:21:09	2019-03-23 20:27:24	1	1.60	7.0	2.15	0.0	12.95	yellow	credit card	
1	2019-03-04 16:11:55	2019-03-04 16:19:00	1	0.79	5.0	0.00	0.0	9.30	yellow	cash	
2	2019-03-27 17:53:01	2019-03-27 18:00:25	1	1.37	7.5	2.36	0.0	14.16	yellow	credit card	
3	2019-03-10 01:23:59	2019-03-10 01:49:51	1	7.70	27.0	6.15	0.0	36.95	yellow	credit card	
4	2019-03-30 13:27:42	2019-03-30 13:37:14	3	2.16	9.0	1.10	0.0	13.40	yellow	credit card	
...	...	...	...	...	...	...	...	...	...	...	
6428	2019-03-31 09:51:53	2019-03-31 09:55:27	1	0.75	4.5	1.06	0.0	6.36	green	credit card	
6429	2019-03-31 17:38:00	2019-03-31 18:34:23	1	18.74	58.0	0.00	0.0	58.80	green	credit card	
6430	2019-03-23 22:55:18	2019-03-23 23:14:25	1	4.14	16.0	0.00	0.0	17.30	green	cash	
6431	2019-03-04 10:09:25	2019-03-04 10:14:29	1	1.12	6.0	0.00	0.0	6.80	green	credit card	
6432	2019-03-13 19:31:22	2019-03-13 19:48:02	1	3.85	15.0	3.36	0.0	20.16	green	credit card	

6433 rows × 14 columns

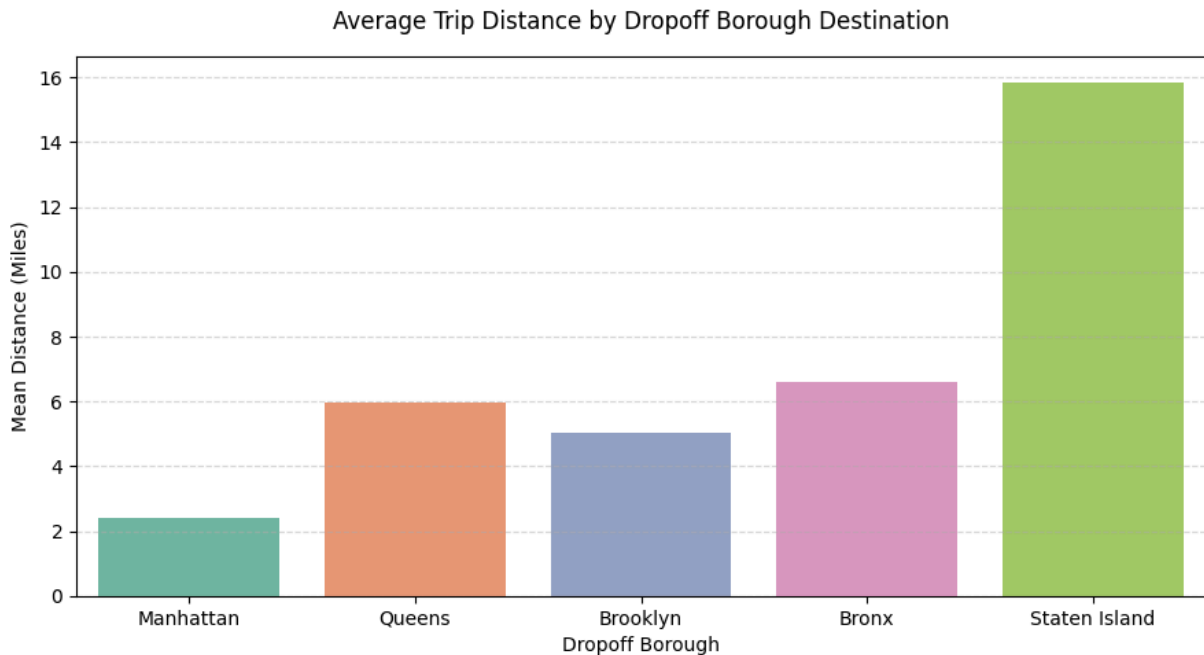


 Barplot

In [23]:

```
plt.figure(figsize=(9, 5))
clean_boroughs = df[df["dropoff_borough"].notna()]
sns.barplot(data=clean_boroughs, x="dropoff_borough", y="distance", hue="dropoff_bo
plt.title("Average Trip Distance by Dropoff Borough Destination", y=1.03)
plt.xlabel("Dropoff Borough")
plt.ylabel("Mean Distance (Miles)")
```

```
plt.grid(True, axis="y", linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()
```



This code creates an aggregated bar plot to compare the average trip distance driven across different dropoff borough destinations without displaying error bars.

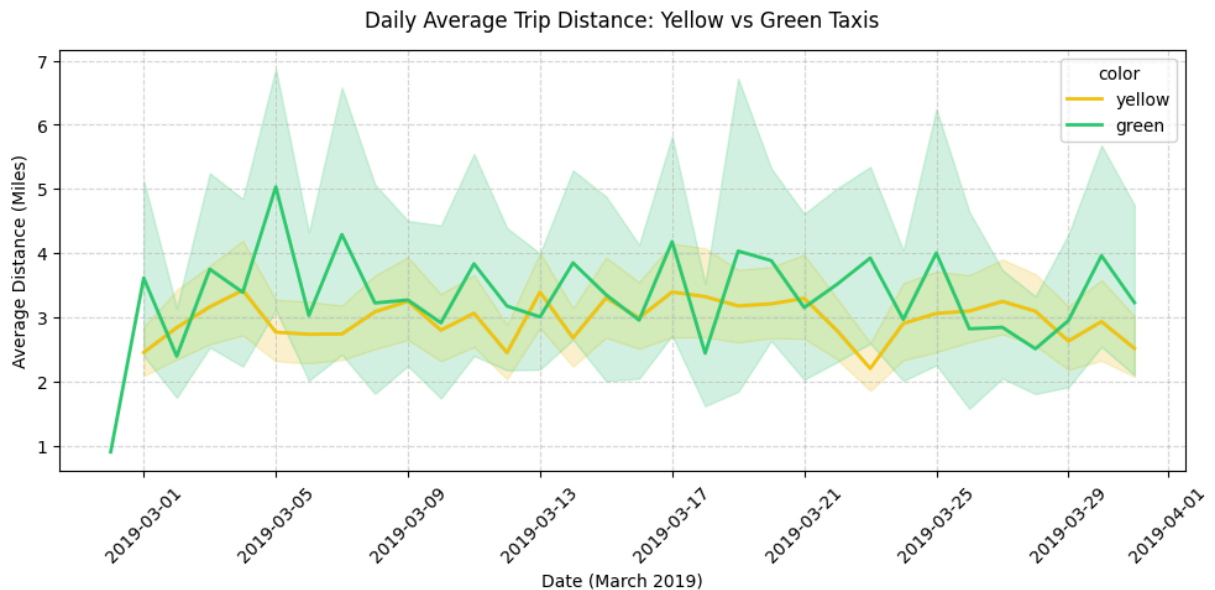
In []:



Lineplot

```
In [26]: df["pickup"] = pd.to_datetime(df["pickup"])
df["pickup_date"] = df["pickup"].dt.date
df["pickup_hour"] = df["pickup"].dt.hour

plt.figure(figsize=(10, 5))
sns.lineplot( data=df, x="pickup_date", y="distance", hue="color", palette={"yellow", "green"})
plt.title("Daily Average Trip Distance: Yellow vs Green Taxis", y=1.03)
plt.xlabel("Date (March 2019)")
plt.ylabel("Average Distance (Miles)")
plt.xticks(rotation=45)
plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()
```



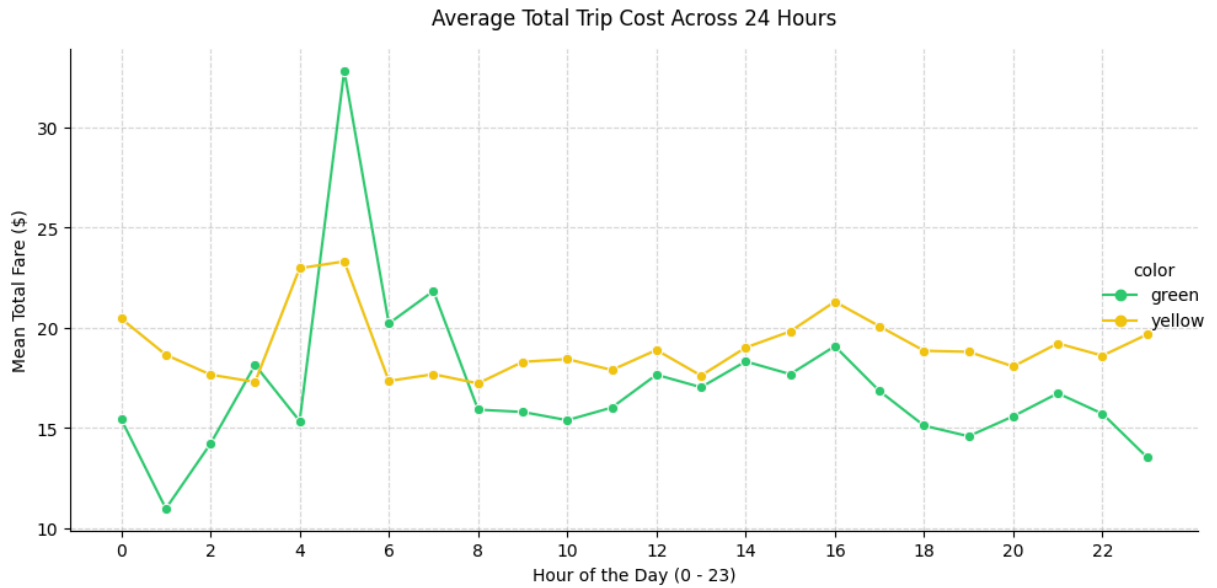
This code creates a time-series line plot to analyze and compare the daily average trip distances traveled by yellow and green taxis over time.

In []:



Relplot

```
In [28]: hourly_fare = ( df.groupby(["pickup_hour", "color"])["total"].mean().reset_index())
sns.relplot(data=hourly_fare,x="pickup_hour",y="total",hue="color",kind="line",
            marker="o", palette={"yellow": "#f1c40f", "green": "#2ecc71"},height=5
plt.title("Average Total Trip Cost Across 24 Hours", y=1.03)
plt.xlabel("Hour of the Day (0 - 23)")
plt.ylabel("Mean Total Fare ($)")
plt.xticks(range(0, 24, 2))
plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()
```



This code creates an hourly relational line plot to compare the fluctuation of average total trip costs across a 24-hour cycle between yellow and green taxis.

In []:

In []:

5 Planets 🪐

```
In [30]: # Load the dataset first (This defines 'df')
df = sns.load_dataset("planets")
df
```

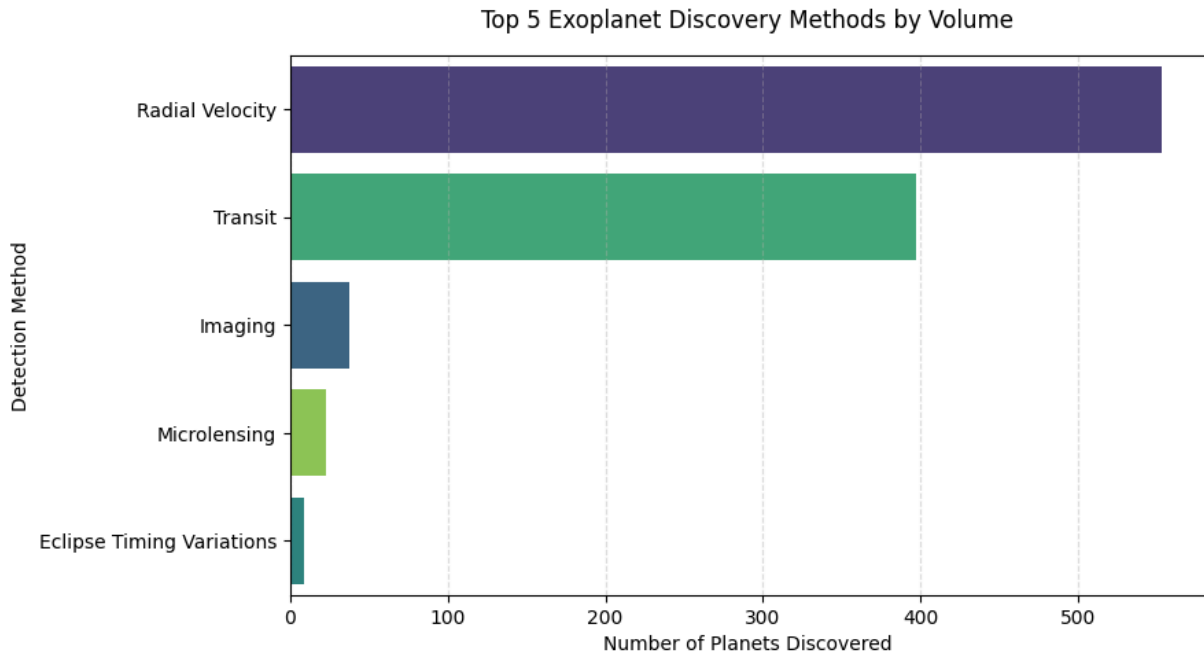

Out[30]:

	method	number	orbital_period	mass	distance	year
0	Radial Velocity	1	269.300000	7.10	77.40	2006
1	Radial Velocity	1	874.774000	2.21	56.95	2008
2	Radial Velocity	1	763.000000	2.60	19.84	2011
3	Radial Velocity	1	326.030000	19.40	110.62	2007
4	Radial Velocity	1	516.220000	10.50	119.47	2009
...	...	...	...	...	...	...
1030	Transit	1	3.941507	NaN	172.00	2006
1031	Transit	1	2.615864	NaN	148.00	2007
1032	Transit	1	3.191524	NaN	174.00	2007
1033	Transit	1	4.125083	NaN	293.00	2008
1034	Transit	1	4.187757	NaN	260.00	2008

1035 rows × 6 columns

Countplot

```
In [32]: plt.figure(figsize=(9, 5))
top_methods = df["method"].value_counts().index[:5]
filtered_df = df[df["method"].isin(top_methods)]
sns.countplot( data=filtered_df, y="method", hue="method", order=top_methods, pale
plt.title("Top 5 Exoplanet Discovery Methods by Volume", y=1.03)
plt.xlabel("Number of Planets Discovered")
plt.ylabel("Detection Method")
plt.grid(True, axis="x", linestyle="--", alpha=0.4)
plt.tight_layout()
plt.show()
```

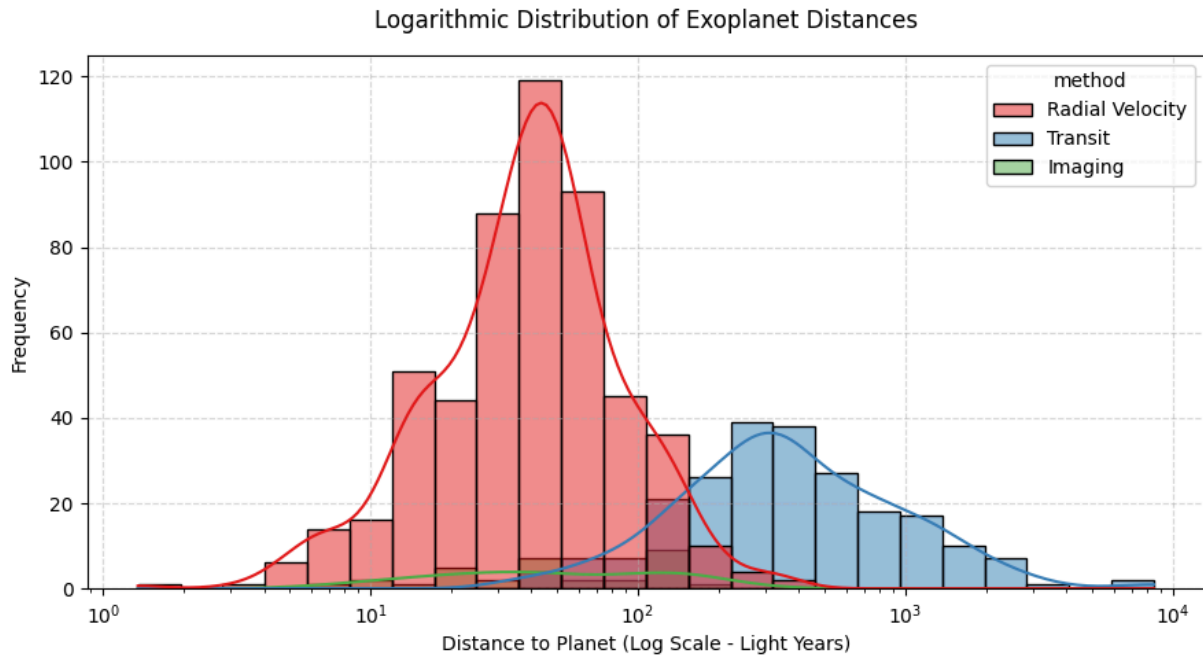


This code creates a horizontal count plot to compare the total volume of exoplanets discovered by the top five most successful detection methods.

In []:

Histplot

```
In [33]: plt.figure(figsize=(9, 5))
sns.histplot(data=df, x="distance", hue="method", hue_order=["Radial Velocity", "Transit", "Imaging", "Microlensing", "Eclipse Timing Variations"],
             multiple="layer", alpha=0.5, log_scale=True, palette="Set1",)
plt.title("Logarithmic Distribution of Exoplanet Distances", y=1.03)
plt.xlabel("Distance to Planet (Log Scale - Light Years)")
plt.ylabel("Frequency")
plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()
```

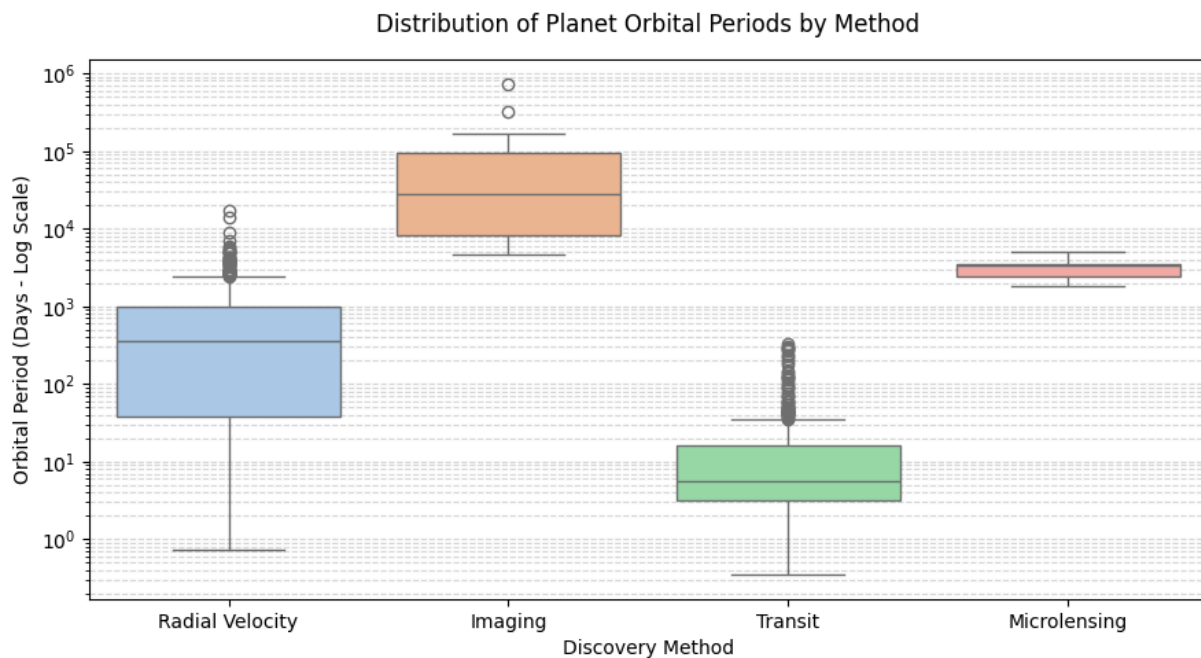


This code creates a log-scaled layered histogram with trend lines to compare the spatial distance distributions of exoplanets discovered via Radial Velocity, Transit, and Imaging.

In []:

Boxplot

```
In [36]: plt.figure(figsize=(9, 5))
top_4_methods = ["Radial Velocity", "Transit", "Imaging", "Microlensing"]
df_top_4 = df[df["method"].isin(top_4_methods)]
sns.boxplot(data=df_top_4, x="method", y="orbital_period", hue="method", legend=False,
plt.yscale("log")
plt.title("Distribution of Planet Orbital Periods by Method", y=1.03)
plt.xlabel("Discovery Method")
plt.ylabel("Orbital Period (Days - Log Scale)")
plt.grid(True, axis="y", linestyle="--", alpha=0.5, which="both")
plt.tight_layout()
plt.show()
```



This code creates a log-scaled box plot to compare the distribution, median ranges, and outliers of planet orbital periods across the top four exoplanet discovery methods.

In []:

In []: